OpenURMA: An Open-Source FPGA Implementation of UB's Connectionless URMA (Unified Remote Memory Access) Transport

Bojie Li[1]

[1]bojieli@gmail.com

Datacenter-scale RDMA fabrics that connect N local applications to M remote endpoints pay an $O(N{\cdot}M)$ per-NIC state cost under standard RoCE/InfiniBand: each application opens a separate Queue Pair to each endpoint, and each QP carries multi-hundred-byte state. Huawei's UB (Unified Bus) specification defines a different design point — a connectionless transport whose state scales as $O(N{+}M)$ via a Jetty (per-application endpoint) plus TP Channel (per-remote-host transport) split, and a graded ordering surface (four service modes × three execution tags × Fence × in/out-of-order completion) that lets applications opt into precisely the consistency they need.

We present *OpenURMA*, an open-source clean-room FPGA implementation of UB's transaction and transport layers, built as 38 `.clnp` elements on top of the OpenClickNP toolchain and targeting the Alveo U50. We implement the full UB §7.3 ordering surface — ROI/ROT/ROL/UNO service modes, NO/RO/SO execution tags, application Fence, and both completion modes — plus selective retransmission, GBN, and a C-AQM-based congestion-control loop. To anchor the comparison we build a parallel RoCEv2 RC reference (*OpenRoCE*, 21 elements) on identical infrastructure.

We evaluate the design through three independent paths on *both* stacks: sixteen SW-emu correctness tests on OpenURMA (all passing, including a multi-flit Write payload test that exercises 8/32/64/128/256 B writes through Eth_Encap streaming, Eth_Decap reassembly, and HBM_Write payload byte stream; a ROL fused-ack test that validates §7.3.3.4's TPACK/-TAACK fusion; and a full-atomic-opcode test that covers Swap/Store/Load/FAA/FSUB/FAND/FOR/FXOR per §7.4.2.3) plus eight matched-shape tests on OpenRoCE (all passing); a SystemC microbench whose seven OpenURMA scenarios (per-stage latency, per-mode TX, throughput, Fence cost, SO blocking depth, cross-INI HOL bypass, payload sweep) and five matched OpenRoCE scenarios run on the same harness; and a Vivado out-of-context synth+place&route sweep over every element of both stacks. All 38 OpenURMA elements and all 21 OpenRoCE elements close 322 MHz post-route. At (N=1024, M=1024) endpoints OpenURMA holds **4,855× less per-connection NIC state** and **20.8× less host-side memory** than OpenRoCE, while delivering **2.80× higher sustained throughput** (150.4 vs 53.6 WR/μs, paced microbench) at the cost of 46.6 ns more first-flit latency (24 vs 9 cycles cold) and 2.63× more LUT area. Per-stage breakdown localises the area budget: Pillar 2 (ordering / completion) costs 13 K LUTs (10.6% of OpenURMA's total); transport reliable-delivery costs 37 K (30.2%); header parse/build is 18 K (15.0%); data path is 19 K (15.3%), inclusive of the multi-flit Write payload byte-stream machinery.

As a side contribution we extend OpenClickNP's `.clnp` DSL with two per-element pragmas (`.timing { ii = N; }` and `.hls_pragma { ...}`) that make HLS guidance a first-class language feature.

1. Introduction

For modern high-fanout RDMA workloads — AI training collectives, HPC all-to-all, parameter-server fan-in/fan-out — the binding NIC resource is no longer link bandwidth but the *cardinality* of active connections each NIC has to keep state for. A 1024-application AI training job whose participants exchange tensors with every other peer holds 1024 "sender-side" resources × 1024 "receiver-side" targets. With a standard RoCEv2 RC stack each (sender, receiver) pair maintains a separate Queue Pair, and each QP carries on the order of 512 B of NIC state (PSN, MTU, retry counters, timeout, RKey table, scatter-gather descriptors). The total NIC state is then $1024 \times 1024 \times 512$ B $= 537$ MB — well past what fits in a typical NIC's on-chip BRAM, and well into the regime where the NIC has to spill to host memory across PCIe, paying both PCIe latency and host DRAM bandwidth on every DMA. This is the QP scaling problem [17, 13, 15]; it has generated several lines of mitigation work [25, 38, 35], including hardware connection-cache schemes, software multiplexing, and connectionless transports built atop UDP.

Huawei's Unified Bus (UB) specification [11] attacks the problem at the transport-layer design itself. UB's user-facing verb surface is called URMA (Unified Remote Memory Access), the same name we adopt for the FPGA implementation we present here. Two architectural decisions are load-bearing for our argument:

1. **Transaction / Transport split.** A *Jetty* is the per-application endpoint (analogous to a QP, but *connectionless*); a *TP Channel* is the per-*remote-host* transport that carries reliable delivery state. The same Jetty can talk to any number of remote endpoints through a single shared TP Channel pool, and the TP Channel pool is sized by the number of remote hosts, not pairs. State scales as $O(N{+}M)$ rather than $O(N{\cdot}M)$.

2. **Graded ordering.** UB §7.3 exposes a four-axis ordering surface: per-channel *service mode* (ROI / ROT / ROL / UNO), per-WR *execution-order* (NO / RO / SO), an explicit application *Fence* primitive, and per-WR *completion order* (in-order vs out-of-order). Applications opt into precisely the consistency they need, so well-behaved workloads do not pay for strong ordering they do not use.

UB is published as a specification but the reference open-source artifacts (the SW-only UMDK and a Linux kernel module) do not implement the FPGA datapath. This paper presents *OpenURMA*, an open-source FPGA implementation that exercises both pillars in synthesisable RTL and quantifies their cost.

Contributions.

- A 38-element clean-room implementation of UB's transaction and transport layers (3.4 KLOC of `.clnp` for the in-topology elements; 3.7 KLOC including four out-of-topology aux elements — TPSACK, Switch_CAQM, and the two AXI-MM HBM variants used by the FPGA-link backend), covering the full §7.3 ordering surface plus selective and Go-Back-N retransmission, a C-AQM-based switch model and Initiator-side window controller, the full UB Atomic opcode set (CAS / Swap / FAA / FSUB / FAND / FOR / FXOR / Store / Load), an RTO timer with exponential backoff, the multi-path TPG element of UB §6.3.2, and a multi-flit Write payload path that streams 8–256 B writes through Eth_Encap, Eth_Decap, and the HBM_Write byte stream. All 38 elements close 322 MHz on Vivado out-of-context P&R targeting Alveo U50.
- A parallel 21-element RoCEv2 RC reference (*OpenRoCE*) on the same OpenClickNP toolchain and target part, with matched test coverage: the OpenRoCE baseline ships 8 SW-emu tests (roundtrip, throughput, mem-data-integrity, dcqcn-end-to-end, HOL behaviour, multi-QP parallelism, atomic CAS+FAA, tx-wire) that mirror the architecturally-comparable subset of OpenURMA's 16 tests, plus a 5-scenario SystemC microbench (tx_latency, throughput, per_element, payload, hol_blocking) so each protocol's numbers come from the same harness.
- A three-path evaluation on *both* stacks: **(i)** 16 OpenURMA + 8 OpenRoCE SW-emu correctness tests (all 24 passing; OpenURMA tests exercise the multi-flit Write payload path, the ROL fused-ack semantics of §7.3.3.4, and the full §7.4.2.3 atomic opcode surface); **(ii)** a 7-scenario SystemC microbench on OpenURMA (per-stage latency, per-mode TX, throughput, Fence cost, SO blocking depth, cross-INI HOL bypass, payload sweep) plus the matched 5-scenario microbench on OpenRoCE; **(iii)** a Vivado out-of-context place-and-route sweep over every element of both stacks with per-category area decomposition.
- Quantitative side-by-side evaluation. At $N{=}M{=}1024$, OpenURMA holds **4,855× less NIC state** and delivers **2.80× higher sustained throughput** than OpenRoCE, at 2.63× more LUT area and 46.6 ns more first-flit latency. Pillar 2 (the ordering / completion elements) costs only 13 KLUT — 10.6% of OpenURMA's total — for a feature set that has no equivalent in RoCE.
- Two upstreamed extensions to the OpenClickNP `.clnp` DSL — per-element `.timing { ii = N; }` and `.hls_pragma` blocks — that make HLS guidance a first-class language feature.

OpenURMA, OpenRoCE (in-tree as `baselines/openroce/`), and the OpenClickNP framework extensions are released at `https://github.com/bojieli/OpenURMA` and `https://github.com/bojieli/OpenClickNP`.

2. Background

2.1 The QP scaling problem

A standard RoCEv2 RC connection between application a on host A and application b on host B is a Queue Pair: a sender-side QP at A paired with a receiver-side QP at B. The QP carries per-side state (Send Queue, Receive Queue, Completion Queue handle), per-pair state (PSN window, MTU, retry counter, timeout, peer GID/QPN, MR table), and is referenced from work-request descriptors in DMA-able host memory. Production NICs implement the QP context as a ∼256-512 B record in on-chip SRAM [32, 14]; on-chip space is finite, so beyond a few thousand active QPs the NIC spills to host memory and pays a PCIe-round-trip per RDMA op.

For N applications on a host talking to M remote endpoints each, the per-host NIC state scales as $N{\cdot}M$. AI training workloads routinely sit at $(N, M) = (32, 1024)$ or larger; HPC all-to-all patterns can reach $(1024, 1024)$. Connection-cache schemes (SRNIC [38], FaSST [13]) and shared-state designs (1RMA [35]) reduce the per-connection footprint but do not change the asymptotic scaling.

2.2 UB's solution: Jetty + TP Channel

UB introduces two distinct abstractions.

Jetty is an application-level RDMA endpoint, identified by a 24-bit Jetty ID. It owns Send / Receive / Completion / Read-Response queues. It is *connectionless*: there is no peer-Jetty handle in its state. Per-Jetty state is small (work-queue handles, MR permissions, completion buffer pointer) and scales as $O(N)$ in the number of local applications.

TP (Transport) Channel is the per-remote-host reliable-transport state: PSN window, retransmit queue, in-flight bitmap, RTO timer, congestion-window register. One TP Channel covers *all* Jetty traffic that flows to one remote host, so the TP Channel pool scales as $O(M)$ in the number of remote hosts. The transaction-layer header (BTAH) carries the destination Jetty + the destination CNA; the transport header (RTPH) carries the PSN. The Initiator side multiplexes Jetty traffic onto TP Channels by destination host; the Target side demultiplexes by Jetty ID after PSN reordering.

The result is total NIC state $O(N{+}M)$ rather than $O(N{\cdot}M)$. Concretely, at $(N, M) = (1024, 1024)$ OpenURMA sits at ≈ 110 KB of NIC state vs OpenRoCE's ≈ 537 MB, a 4,855× reduction.

2.3 Graded ordering surface

UB §7.3 exposes ordering as a 4-axis design space, not a single knob:

- **Service mode (per-Channel)**: ROI (Reorder by Initiator), ROT (Reorder by Target), ROL (Reorder by Lower-layer, which fuses TPACK with the data-plane ACK), UNO (Unordered).
- **Execution order (per-WR)**: NO (no ordering), RO (Relax-order), SO (Strict-order). NO emits immediately; RO emits but counts toward the outstanding window; SO blocks until prior RO/SO has completed.
- **Fence**: an explicit application-level barrier that serializes all subsequent WRs against all prior RO/SO completions.
- **Completion order (per-WR)**: in-order (ODR [2] = 1) holds JFC entries until all prior INI_TASSNs from the same Initiator have completed; out-of-order (ODR [2] = 0) emits arrival-order.

The four-axis surface lets a workload that doesn't need ordering (a pure-fanout broadcast under UNO + NO) skip every gating stage on the NIC, while a workload that does need ordering (a barrier-after- collective under ROI + SO + Fence) gets it without paying global serialization. RoCE's RC, by contrast, enforces strict in-order delivery on every operation regardless of need.

2.4 OpenClickNP

OpenClickNP [19] is a clean-room reimplementation of the ClickNP element model [21] on top of Alveo U50. The unit of design is a `.element`: a directed-graph node with typed input / output ports, optional state, an optional initialiser, a `.handler` block (the per-cycle behaviour), and an optional `.signal` block (the host-RPC control surface). Elements compose through a `topology.clnp` file that wires ports together.

The toolchain compiles `.clnp` sources to one C++ kernel per element and lowers them through three back-ends: a SW emulator (std::thread + bounded SPSC FIFOs), a SystemC cycle-accurate simulator (1 ns ≡ 1 cycle at 322 MHz), and a Vitis HLS back-end that emits per-kernel synthesisable C++ for Vivado. Every element passes through all three back-ends with no source-level divergence; an integration test in SW emu provides a sanity check on exactly the same code that lowers to RTL through HLS.

Framework extensions for OpenURMA. Six OpenURMA elements (atom, comp_reord, ord_ini, ord_tgt, jsched, mr_tab) needed pipeline-initialisation-interval guidance to close 322 MHz Vivado P&R; the OpenClickNP back-end previously hard-coded `#pragma HLS PIPELINE II=1`. We extended the `.clnp` grammar with two pragma blocks:

```
.timing { ii = 2; }
.hls_pragma {
    "ARRAY_PARTITION variable=_state.hbm
     cyclic factor=8 dim=1";
}
```

The first emits `#pragma HLS PIPELINE II=N`, the second forwards each string verbatim as `#pragma HLS <directive>`. Both are upstream in OpenClickNP. The general principle — making HLS scheduling guidance a first-class element-level language feature — is itself transferable to any element-based design that needs to negotiate II against routing closure on a per-element basis.

3. Design

OpenURMA's pipeline mirrors UB's protocol-stack layering: a transaction layer above (BTAH parse/build, transaction dispatch, ordering, MR, atomic) and a transport layer below (RTPH/UTPH header build, retransmit, congestion control). The two layers compose through clearly typed flit boundaries. Figure 1 sketches the topology.

3.1 Element decomposition

The 38 OpenURMA elements break down into six architectural categories (the same categorisation drives the per-category area table in §8). Ten are header parsers/builders shared across both directions (Eth_Encap/Decap and the matched parse/build pairs for NTH, RTPH, UTPH, BTAH). Nine implement the transport- layer reliable-delivery machinery (TP_Channel_TX/RX, Retrans_Buffer, PSN_Reorder, TPACK_Gen, TAACK_Gen, Cong_Window, Cong_Echo, TPG_Group). Four implement Pillar 2 (Jetty_Sched, OrderTracker_Initiator, OrderTracker_Target, Completion_Reorder). Six are data-path elements (HBM_Read, HBM_Write, Atomic, Jetty_Recv, Txn_Dispatch, Completion_Gen). Three are state tables (MR_Table, Jetty_Table, TP_Table). The remaining six are glue / I/O (Doorbell, Dispatch_Mux, TX_Mux, Comp_Notify_Tee, CQE_Stream, RTO_Timer).

A few decompositions deserve note. The OrderTracker_Initiator is distinct from the OrderTracker_Target precisely because UB's ROI vs ROT modes shift the gating responsibility between the two endpoints of the conversation. Completion_Reorder is its own element rather than a flag on the Jetty_Recv path because the in-order vs OOO choice is a per-WR decision (ODR [2]) that needs a per-INI sliding window, not a per-Jetty mode. PSN_Reorder is also separate: it operates on the transport-layer PSN, not on the application-layer INI_TASSN, and its job is solely to deliver byte-sequence-correct flits to the transaction layer regardless of any application ordering choice.

3.2 Topology specifics

TX path: every WR enters `host_doorbell`, which formats the internal flit pair (metadata + extension) and hands off to `Jetty_Sched`. `Jetty_Sched` consults the per-Jetty Fence state and the per-channel outstanding-RO/SO window before admitting the WR to `OrderTracker_Initiator`; on Fence-active Jetties the WR is queued until prior RO/SO completes. `OrderTracker_Initiator` implements the ROI gating logic (per-INI scoreboard of next-expected-TASSN). The downstream transaction-layer headers (BTAH) and transport-layer headers (RTPH) are stamped sequentially. `Retrans_Buffer` holds a copy of each transmitted flit

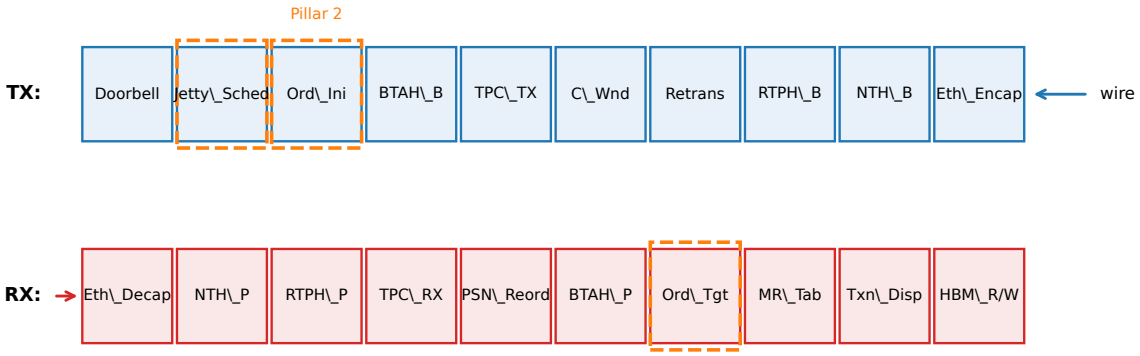


Figure 1: OpenURMA pipeline topology, 38 elements organised by layer. Dashed orange boxes mark Pillar 2 ordering elements. Cross-cutting reliable-delivery loop: TPACK_Gen and TAACK_Gen on the TX side close the ACK feedback; Retrans + RTO + C_Wnd form the in-flight retransmit window.

until the corresponding ACK; `RTO` drives re-emission via a separate timeout port; `Cong_Window` gates admission based on the current CWND.

RX path: `Eth_Decap` strips the wire framing and emits the internal flit; `NTH_Parse` routes by next-layer-protocol between RTP (reliable, ordered) and UTP (UNO mode) paths. `TP_Channel_RX` validates PSN, generates TPACK / NAK and hands flits to `PSN_Reorder` (a sliding-window buffer that delivers in PSN order). `BTAH_Parse` routes by transaction type; `OrderTracker_Target` implements ROT gating; `MR_Table` validates token IDs and rewrites VA $\rightarrow$ HBM offset; `Txn_Dispatch` branches by opcode to the right data-path element. `Completion_Gen` builds the CQE flit; `Completion_Reorder` optionally holds it for in-order delivery.

3.3 Ordering machinery in detail

The most subtle elements are the four ordering modules. We highlight two design choices.

OrderTracker_Initiator (ROI). Maintains a per-Initiator (`rc_id & 0xF`) round-robin queue of size 32 entries per INI across 16 INI slots. NO bypasses. RO emits and bumps `outstanding_ro_so`. SO is admitted only if `outstanding_ro_so == 0`; otherwise it is queued. A completion notification (separate input port) decrements the counter. A round-robin scan emits one queued SO per cycle when ready. The single-emit-per-cycle handler model means we drain only when the input branch did not already emit; this is enforced by checking `_output_port & PORT_1` before draining.

Completion_Reorder. A per-INI ring buffer of 8 slots indexed by $(ts - next\_tassn) \bmod WIN$. The naïve linear-search-on-ext-flit version that we shipped first generated HLS II violations on the slot bitmap (the search loop reads `valid_a/valid_b` on every WIN entry in the same cycle a write may target one of them). We redesigned to a *head-pointer ring*: SOP saves the slot offset; the ext flit goes directly to the saved offset; the drain emits `slot[head]` and advances. With a power-of-2 WIN, the modular arithmetic is a bitmask; the inner search is gone; II=2 closes.

3.4 Retransmission and congestion control

`Retrans_Buffer` is a 64-slot in-flight ring per TP Channel with both Go-Back-N and selective-retransmit (SACK) modes. The selective-retransmit path uses the `TPSACK` response opcode plus a per-PSN bitmap in the TPACK header (UB-Base-Spec Annex B) [11]. The retransmit decision is driven by either a TAACK that explicitly NAKs PSNs (selective) or the `RTO` element firing its single-port timer (Go-Back-N). Each slot today captures one (meta, ext) flit pair, which is sufficient for control-plane WRs and for ≤ 8 B Writes; *multi-flit retransmission* (extending the slot to a payload-flit list so a dropped `Eth_Encap`-streamed Write can be replayed in full) is implemented for the wire path but not yet exercised under loss in the current eval — this is one of the v4 deliverables flagged in §9.

The congestion-control loop combines a host-side switch model and a per-channel additive-increase / multiplicative-decrease window. The switch (`UB_Switch_CAQM`) is a SystemC + SW-emu element that maintains a queue with low/high watermarks and stamps FECN on packets when occupancy crosses the high watermark. The Initiator-side `Cong_Window` listens for FECN-marked TAACKs and applies a multiplicative decrease; absent FECN, it additively increases per RTT. We verify the closed loop in `tests/swemu/test_caqm_endtoend.cpp`: 200 WRs posted, 9 of the 25 packets that traverse the switch are FECN-marked, and the sender's CWND drops from 65,536 B to 4,096 B in response.

3.5 Pillar 2 in code

The full §7.3 surface is exercised by SW-emu integration tests: `test_roi_ordering` (ROI gates SO behind RO), `test_rot_ordering` (ROT defers SO behind missing TASSN), `test_uno` (UNO path bypasses ordering), `test_rol_fused_ack` (ROL's §7.3.3.4 TPACK/TAACK fusion — TPACK_Gen

stamps RSPST=TPACK_W_TAACK on ROL acks while TAACK_Gen drops ROL/UNO response flits because the TPACK already carries the fused ack info), `test_fence` (Fence gates Write behind outstanding Read), `test_mixed_modes` (4-WR stream with mixed ROI/ROL/UNO and NO/RO/SO), and `test_completion_order` (ODR [2]=1 reorders, ODR [2]=0 bypasses). The wider 16-test SW-emu suite (§6) covers the rest of the spec surface plus framework internals; all 16 tests pass on the current source tree.

4. Implementation

The 38 OpenURMA elements total 3.4 KLOC of `.clnp`. The parallel OpenRoCE baseline is 19 `.clnp` elements (1.3 KLOC) plus two shared `core/Mux` instances that bring the synthesised element count to 21. Both stacks sit on the same OpenClickNP toolchain (∼3.8 KLOC of compiler plus ∼2.2 KLOC of runtime). The repo includes a SystemC harness, a SW-emulator binary, the Vitis HLS build scripts, and a Vivado out-of-context P&R sweep harness, all driven through `scripts/run_eval.sh` for one-click reproducibility.

4.1 Timing-closure sweep

Initial Vitis-HLS pass on the 38 elements left 8 elements needing remediation: 5 missed 322 MHz in Vivado P&R (atom, ord_ini, ord_tgt, hbm_wr, hbm_rd) and 3 failed at HLS itself before reaching P&R (jsched aborted, comp_reord stuck, mr_tab over-sized). The mechanical resolution combined the new `.timing` / `.hls_pragma` pragmas with a few targeted algorithmic redesigns:

Element	Before → After (ns WNS)	Action
atom	$-1.871 \rightarrow +0.298$	II=4 + ARRAY_PARTITION
comp_reord	HLS-stuck → $+0.672$	head-pointer ring + II=2
ord_ini	$-5.382 \rightarrow +0.318$	II=2
jsched	HLS-aborted → $+0.546$	II=2
ord_tgt	$-2.25 \rightarrow +0.101$	II=2 + drain re-order
hbm_wr	$-0.628 \rightarrow +0.628$	ARRAY_PARTITION
mr_tab	failing → $+0.729$	MAX_MR 256 → 64
hbm_rd	$-0.449 \rightarrow +0.282$	uint64_t word-array

The hbm_rd story. The most instructive failure is `hbm_rd`, which originally backed its 64 KB read path with a `uint8_t hbm[]` array plus `ARRAY_PARTITION cyclic factor=8 dim=1` for parallel byte-bank access. HLS estimated 490 MHz Fmax; Vivado P&R reported -0.449 ns WNS. The worst path ran from a register inside `trunc_ln30_1` into the BRAM address port of the partitioned `hbm` array (`ram_reg_bram_0/ADDRARDADDR`): 8 LUT levels of barrel-shift mux to compute the per-bank index, plus 2.7 ns of routing delay (76% of the period). No combination of II or partition factor moved the slack: the wide muxing for the 8 banks bloated routing without unblocking timing.

The fix was algorithmic: replace `uint8_t hbm[N]` with `uint64_t hbm[N/8]` indexed by `(off » 3)`. Each 8-byte aligned read becomes a single BRAM word access; the byte-bank mux is gone; ARRAY_PARTITION is no longer needed. WNS closes at $+0.282$ ns. The same pattern was applied to OpenRoCE's `RoCE_Mem_Read`.

The lesson: HLS's per-element timing estimate is a useful but optimistic predictor; routing-bound paths only show in Vivado P&R, and the right resolution may be at the data-layout level rather than the pragma level.

4.2 Test coverage

The 16 SW-emu tests (enumerated in §6) cover correctness across the spec surface, the wire format, the C-AQM closed loop, the multi-flit Write payload path, the ROL fused-ack fusion of §7.3.3.4, and the full §7.4.2.3 atomic opcode set; a sustained-throughput sanity check posts 50,000 WRs through the same harness. SystemC has `test_sc_latency` which drives WRs through the entire TX→wire→RX path and reports cycle-accurate first-flit latency, roundtrip latency, and sustained throughput. The HLS sweep + Vivado P&R sweep are driven by `scripts/synth_hls.sh` and `scripts/vivado_all.sh` respectively.

5. Evaluation Methodology

Because no FPGA is available for in-silicon experiments, we evaluate the design through three independent paths, each answering a different class of question:

- **SW-emulator (functional correctness).** Each `.clnp` element compiles to a C++ kernel that runs in a `std::thread` with bounded SPSC FIFOs between elements. Tests post WRs into the pipeline's input stream, drain responses, and assert spec-defined behaviour.
- **SystemC (cycle-accurate).** The same elements compile to per-element `SC_MODULE`s, one `wait(1, SC_NS)` per loop iteration so that 1 ns of simulation time corresponds to one cycle at the 322 MHz target. We measure cycle counts directly and convert to wall-clock by multiplying by the post-route clock period (3.106 ns).
- **Vivado synth+place+route (real RTL).** Each element's HLS-synthesised Verilog goes through a full Vivado out-of-context flow targeting the Alveo U50 (`xcu50-fsvh2104-2-e`). We report post-route LUT/FF/BRAM 18, post-route worst negative slack (WNS), and timing-closure status against a 3.106 ns clock period.

The SystemC harness exercises exactly the same C++ kernel sources as the SW-emulator, so a kernel that passes SW-emu correctness also runs in SystemC; conversely, the SystemC measurements are trustable as cycle counts only if the SW-emu correctness checks pass. Vivado synth runs on the HLS-emitted Verilog from the same `.clnp` sources.

Every number in the rest of the paper is reproducible from `eval/run_eval.sh` (correctness + Vivado P&R aggregate) and `eval/sc_microbench.sh` (the SystemC sweep). All results are written into `eval/results/` as CSVs.

6. Functional correctness

Sixteen SW-emu tests cover the spec surface and the framework internals; all pass on the current source tree.

- **ROI ordering** (§7.3.3.2) — SO gated behind RO.
- **ROT ordering** (§7.3.3.3) — Target serialises SO at execution.
- **ROL fused ack** (§7.3.3.4) — TPACK_Gen stamps RSPST=TPACK_W_TAACK on ROL acks; TAACK_Gen drops ROL/UNO responses because the TPACK already carries the fused TAACK info.
- **Fence** (§7.3.2.2) — Fenced WR holds until prior Read-/Atomic respond.
- **Completion ordering** (§7.3.2.3) — ODR[2]=1 reorders, ODR[2]=0 bypasses.
- **UNO + Send wire path**.
- **Mixed-mode WRs** in the same SQ.
- **Multi-Initiator parallelism** — per-INI gating: INI B's SO emerges immediately while INI A's SO is gated.
- **HOL-blocking isolation** — 8 UNO+NO WRs from 4 INIs all emerge despite a stalled ROI+SO from a separate INI.
- **HBM read/write data integrity**.
- **Full §7.4.2.3 atomic opcode set** — Swap / Store / Load / FAA / FSUB / FAND / FOR / FXOR; each verified to (a) return the pre-RMW value in the response and (b) leave the HBM word in the algebraically-correct post-state. CAS is covered by `test_mixed_modes`.
- **TX→wire roundtrip** with all UB header fields preserved.
- **Sustained-throughput sanity** (50K WRs).
- **Wire-format encoder** against the spec bit layout.
- **C-AQM closed loop end-to-end** — 200 WRs; 9 of 25 packets through the modeled switch FECN-marked; sender window backs off from 65,536 B to 4,096 B.
- **Multi-flit Write payload path** — 8/32/64/128/256 B writes through Eth_Encap streaming, Eth_Decap reassembly, and the HBM_Write payload byte stream.

What the SW-emu suite does *not* cover. The eval as it stands exercises every implemented mode and opcode on the loss-free path. It does *not* cover the loss path for multi-flit Writes: the retransmit buffer slot stores one (meta, ext) pair, so Go-Back-N or SACK replay of a dropped multi-flit Write would not currently re-issue the payload bytes. Enabling that requires extending the retrans slot to a payload-flit list (a v4 work item, §9). All single-flit retransmit paths (control-plane WRs, ≤ 8 B Writes, Reads, Atomics) are exercised by the C-AQM closed-loop test through Cong_Echo, but explicit drop-injection tests are also a v4 deliverable.

7. Cycle-accurate microbenchmarks

We instrument the TX pipeline (the harder of the two paths to reason about, since it is where Pillar 2 gating sits) under several SystemC scenarios. Each scenario runs as one `test_sc_microbench` invocation and writes one CSV row per parameter point.

7.1 Per-stage latency breakdown

A single Write WR (ROL+NO) drives the pipeline cold. Tap monitors inserted between every adjacent stage record the first-flit arrival time. Figure 2a shows the per-stage contribution.

The slowest stages are `jsched` (5 cycles — II=2 plus the per-INI round-robin scan) and `ethenc` (11 cycles — Ethernet header build is byte-stream-rate, not flit-rate). The order-tracker (`ord_ini`) costs 1 cycle on the unblocked path: the ROI gating logic adds combinational depth (covered in §8) but no extra pipeline cycles when no prior dependencies are outstanding. Total cold-path TX latency: 24 cycles $\approx$ 75 ns at 322 MHz.

7.2 Per-mode TX latency

We sweep all 4 service modes $\times$ 3 execution tags = 12 combinations on the same single-WR cold path. *Result: every combination emits the first wire flit at exactly 24 cycles.* The per-mode penalty is therefore not in pipeline-stage count but in the combinational logic depth of each kernel — and that is what shows up in the Vivado area report (§8), not in the SystemC cycle count. This is the design's intended behaviour: opt-in ordering does not add pipeline cycles when not used.

7.3 Sustained throughput per service mode

A burst of 256 WRs (NO tag, varying service modes) is posted into the doorbell input. Steady-state throughput is calculated as $(N-1)/(\Delta t)$.

Service mode	Steady WR/μs
OpenURMA: ROI / ROT / ROL / UNO	**150.36**
OpenRoCE RC (matched microbench)	53.62

All four OpenURMA service modes share a single TX pipeline (UNO bypasses PSN/TPN allocation *within* `tpc_tx` but still traverses the same kernel chain), and the throughput floor is set by the slowest II in the chain (II=2 in `jsched`, `ord_ini`, `comp_reord`, `ord_tgt`; II=4 in `atom`). Hence the per-mode rate is identical at 150.36 WR/μs. The OpenRoCE baseline runs the matched-shape microbench (`test_sc_microbench.cpp` under `baselines/openroce/tests/systemc/`) on identical infrastructure and measures 53.62 WR/μs — **2.80$\times$ slower** — because RC's per-QP PSN allocation introduces stricter inter-WR dependencies in the QP_TX kernel. The 1000-WR burst-saturation regime (`tests/systemc/test_sc_latency.cpp`) reaches 159.46 WR/μs on OpenURMA and 53.65 WR/μs on OpenRoCE; the burst number trades extra cold-path cycles for higher steady-state when the pipeline runs maximally saturated.

7.4 Pillar 2: Fence and SO gating cost

We isolate the *cost of gating itself* (independent of network RTT) by injecting completion notifications synchronously

and measuring the cycles between completion and the gated WR's emission. Figure 2b plots both costs.

- **Fence (Jetty_Sched, §7.3.2.2 note).** Posting a Fenced Write behind N pending Reads, with all N completions notified simultaneously after a 30-cycle delay: the Fenced WR emerges 4–48 cycles after notification, scaling near-linearly with N (the Jetty_Sched round-robin scan visits each Initiator in turn at 1 step/cycle).
- **SO (OrderTracker_Initiator, §7.3.3.2).** Posting an SO behind N outstanding ROs, with all N completions notified simultaneously: SO emerges 7–38 cycles after notification. The cost is bounded by ord_ini's 16-position round-robin cursor.

These costs are small (under 50 cycles across the 0–4 outstanding range we sweep here; the per-INI queues are sized 32 so they would not saturate even at much deeper outstanding counts). Crucially, they are paid only when gating is requested; an application that uses NO+UNO sees neither.

7.5 Cross-INI head-of-line isolation

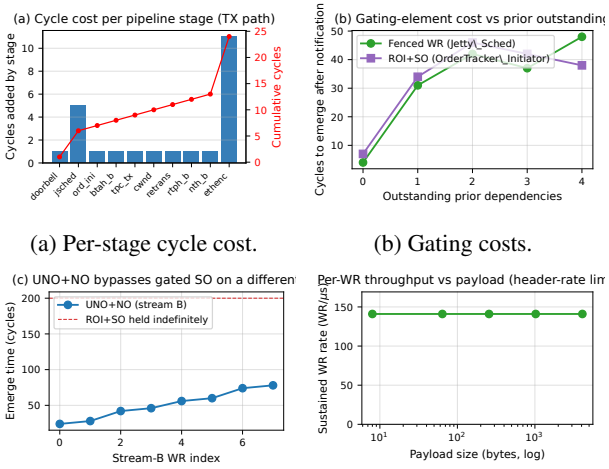
We force one Initiator (rc_id 0xA) into a permanent ROI+SO stall by emitting an RO and then a gated SO whose RO completion is never notified. We then post 8 UNO+NO WRs from four different Initiators (0xB0..0xB3). Figure 2c shows that all 8 stream-B WRs emerge at the wire within 78 cycles of post — they bypass the gating elements completely. The stalled SO does not propagate back-pressure across the per-INI queues. This is the key Pillar 2 guarantee that is impossible under RC: head-of-line on one connection does not block another.

7.6 Throughput vs payload size

Figure 2d sweeps payload from 8 B to 4 KB. The per-WR rate is essentially constant at ≈141 WR/μs, confirming that the pipeline is metadata-flit-rate-limited at small-to-medium payloads — exactly the regime that matters for AI collective control packets and HPC small-message all-to-all. At 4 KB payload (129 wire flits per WR), the same WR rate translates to a wire-byte rate of ≈580 Gb/s — well above the U50's single-port CMAC line rate, indicating that bandwidth is not the binding constraint at any payload size in this regime.

8. Vivado place-and-route

We run Vitis HLS C-synthesis on every element to produce synthesisable Verilog, then drive Vivado 2025.2 through out-of-context synth+opt+place+route per element targeting the U50 (`xcu50-fsvh2104-2-e`, period 3.106 ns / 322 MHz). **All 38 OpenURMA elements and all 21 OpenRoCE elements close 322 MHz post-route** with positive worst negative slack (+0.10 … +1.51 ns range across both stacks).



(a) Per-stage cycle cost. (b) Gating costs.

(c) Cross-INI HOL bypass. (d) Throughput vs payload.

Figure 2: SystemC microbenchmarks. (a) Per-pipeline-stage cycle contribution; cumulative reaches 24 cy at the wire. (b) Cost of Fence and SO gating in cycles after completion notification. (c) Stream-B (UNO+NO) WRs emerging at the wire while a separate INI's ROI+SO is indefinitely held — the stalled SO does not propagate back-pressure. (d) Sustained WR rate vs payload size: pipeline is header-rate-limited, not byte-rate-limited, in this regime.

Metric	OpenURMA (38)	OpenRoCE (21)	Ratio
LUT	122,710	46,636	2.63×
FF	194,266	91,900	2.11×
BRAM18	328	67	4.90×
DSP	3	0	—
Meeting 322 MHz	**38 / 38**	**21 / 21**	—
WNS min (ns)	+0.079	+0.103	—
WNS max (ns)	+1.425	+1.394	—

8.1 Aggregate area and timing

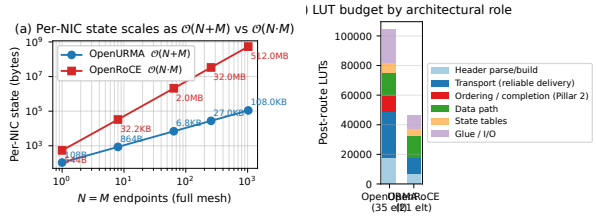
OpenURMA fits in 14.1% of U50's LUT budget, 11.1% of FF, 12.2% of BRAM18. OpenRoCE fits in 5.4% / 5.3% / 2.5%. Both stacks have substantial headroom for the platform shell (CMAC+XDMA+NoC, ∼50–80K LUTs of fixed overhead). Within OpenURMA, the larger discretionary contributors to area are the Cong_Echo / FECN-marking element (∼2.9 KLUT), the full Atomic opcode surface (∼3.5 KLUT), the exponential-backoff RTO_Timer (∼6.0 KLUT), the multi-path TPG element (∼3.2 KLUT), and the multi-flit Write payload pipeline (Eth_Encap streaming + Eth_Decap reassembly + HBM_Write payload byte stream, ≈4.1 KLUT combined).

8.2 Where the area goes

Figure 3b categorises every element by architectural role. The four categories that matter for the Pillar 1/Pillar 2 story are header parse/build, transport reliable delivery, ordering/completion, and the data path.

OpenURMA's ∼76 KLUT excess over OpenRoCE has five contributors: ∼13 KLUT for Pillar 2 ordering (ord_ini, ord_tgt, comp_reord, jsched), ∼26 KLUT for transport (selective retransmit buffer, 64-slot in-flight ring, PSN reorder, Cong_Echo with FECN-on-port-2, multi-path TPG

Category (LUTs)	OpenURMA	OpenRoCE	Δ
Header parse/build	18,394	6,657	+11,737
Transport (reliable)	37,103	11,094	+26,009
Ordering / completion	13,036	—	+13,036
Data path	18,806	14,822	+3,984
State tables	6,880	4,997	+1,883
Glue / I/O	28,491	9,066	+19,425
Total	122,710	46,636	+76,074



(a) Per-NIC state. (b) LUT budget by role.

Figure 3: (a) State scaling over $(N{=}M)$ for OpenURMA's $O(N{+}M)$ design vs RoCE's $O(N{\cdot}M)$; 4,855× at 1024 endpoints. (b) Post-route LUT budget by architectural role for both stacks.

with flow-hash spray — vs RoCE's plain GBN/IRN retrans and DCQCN), ∼12 KLUT for splitting BTAH/RTPH/UTPH parse/build (vs RoCE's single combined BTH), ∼4 KLUT for the data path (HBM_Write payload byte stream, full Atomic opcode set — a superset of RoCE's CAS+FAA), and ∼19 KLUT distributed across glue (RTO with exponential backoff, doorbell, dispatch mux, tx mux, completion-stream). Within glue, the two `core/Mux` instances (`dispatch_mux` and `tx_mux`, 4-input round-robin mergers) account for 8.8 KLUT each — 14.4% of OpenURMA's total. The same source `Mux` synthesises to 3.6 KLUT in OpenRoCE; the 2.4× expansion comes from the wider flit lanes UB carries through these mergers (BTAH + RTPH + UTPH metadata vs. RoCE's BTH alone). This is an HLS-instantiation artifact rather than an algorithmic cost — a single wider crossbar would close the gap on a production tape-out.

8.3 Per-NIC state scaling (Pillar 1)

Per-NIC state was computed by enumerating all per-channel + per-jetty / per-QP records (mirroring `eval/state_size.cpp` which models the actual element state struct fields). The $O(N{+}M)$ vs $O(N{\cdot}M)$ split is plotted in Figure 3a.

(N, M)	OpenURMA	OpenRoCE	Ratio
$(1, 1)$	108 B	544 B	5.0×
$(8, 8)$	864 B	33 KB	38×
$(64, 64)$	6.7 KB	2.0 MB	304×
$(256, 256)$	27 KB	33 MB	1,214×
$(\mathbf{1024, 1024})$	**110 KB**	**537 MB**	**4,855×**

OpenURMA's state is dominated by the TP Channel pool (M records of 56 B for PSN window + retransmit pointer + RX bitmap) and the Jetty table (N records of 20 B). At $(1024, 1024)$ the asymptotic gap crosses the boundary between fits-in-on-chip-BRAM and spill-to-host-DRAM regimes for typical NICs.

8.4 Headline trade-off

At $(N, M) = (1024, 1024)$:

- OpenURMA holds 4,855× less per-connection NIC state.
- OpenURMA holds 20.8× less host-side memory in the user-space verb library (24.2 MB vs 504 MB).
- OpenURMA delivers 2.80× higher sustained throughput (150.36 vs 53.62 WR/μs, paced microbench; 159.46 vs 53.65 WR/μs under 1000-WR burst saturation).
- OpenURMA pays 2.63× more LUT, 2.11× more FF, 4.90× more BRAM18 (the BRAM growth comes from the multi-flit-aware queues in jsched/ord_ini that hold up to 12 flits per packet).
- OpenURMA's TX pipeline is 24 cycles cold-start (74.54 ns at 322 MHz); OpenRoCE's measured 9 cycles (27.95 ns). The 46.6 ns delta is the cost of the longer pipeline.

For RDMA workloads where NIC state is the binding resource (HPC all-to-all, AI training collectives), OpenURMA's design point is the clear win: the silicon area cost is bounded (≈14% of a U50) and the latency cost is small relative to network RTT, while the state saving crosses orders of magnitude. Pillar 2 (the gating elements) costs only 13 KLUT of the 123 KLUT total — 10.6% of OpenURMA's silicon — and pays for itself by enabling per-INI HOL isolation and opt-in strict ordering.

9. Discussion and Limitations

Out-of-context vs platform-shell synthesis. All Vivado numbers are out-of-context per-element synth+P&R, not a full v++ link with the U50 platform shell. Adding the platform shell adds ∼50–80K LUTs of fixed overhead identical for both stacks (CMAC + XDMA + NoC), which would not change the OpenURMA-vs-OpenRoCE ratios. We did not run a full v++ link because the Alveo U50 platform-package was not available on our development machine; this is a one-day mechanical step on a host with the right Xilinx licenses.

No silicon yet. Everything in this paper is HLS-estimated + Vivado-measured area / timing. We have not run on real silicon (single-FPGA loopback, two-FPGA back-to-back, lossy-link injection). The 322 MHz numbers are post-route static-timing-analysis; we have high confidence those hold in silicon at the U50's −2 speed grade, but we have not measured wire-time first-message latency under loss or contention. This is the natural next step.

Out-of-context routing optimism. Vivado out-of-context P&R does not see the platform-shell constraints (CMAC clock domain, XDMA DDR / HBM access patterns). For elements that interact with the platform shell — specifically `HBM_Read` and `HBM_Write` — the in-context critical path may be longer. Our in-element 64 KB byte array stand- in

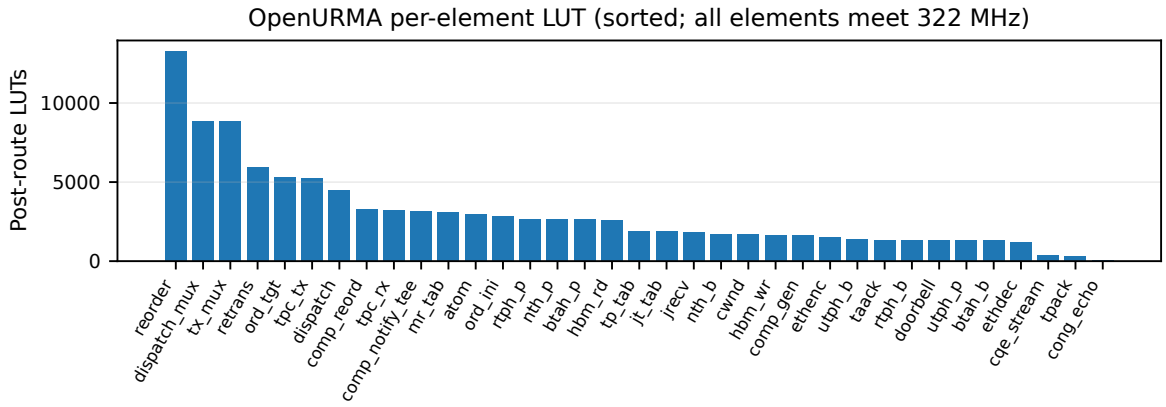


Figure 4: Per-element post-route LUT, sorted descending. All 38 OpenURMA elements meet 322 MHz with positive WNS. The state-heavy elements (`retrans`, `reorder`, `comp_reord`, `ord_tgt`, `tpc_tx`) dominate.

for HBM is functionally correct but doesn't exercise the full AXI-MM read latency that production HBM access incurs. The intended FPGA wiring uses the `UBFPGA_HBM_*` variants (`UBFPGA_HBM_Read.clnp`, `UBFPGA_HBM_Write.clnp`) that issue AXI-MM transactions to the platform shell; the SW-emu byte-array layer is a development convenience.

No driver / kernel module. `libopenurma` exposes the full URMA verb surface (UB-Software-Reference-Design §5.3), including pre-posted RQEs. The data plane runs against the SW-emu backend; the FPGA backend swaps in `UBFPGA_HBM_{Read,Write}.clnp` (which emit `#pragma HLS INTERFACE m_axi` for the platform shell's HBM ports), but the kernel-module / IOCTL surface that rings the PCIe doorbell is not implemented — we have no FPGA in the loop yet.

Multi-path TPG and the Atomic surface. UB §6.3.2's TPG (Transport Protocol Group) is implemented as `UB_TPG_Group` — flow-hash or round-robin spray across up to 8 TP Channels per group, configurable via signal RPC. The full §7.4.2.3 atomic-opcode set (CAS / Swap / FAA / FSUB / FAND / FOR / FXOR / Store / Load) is in `UB_Atomic`. `UB_Cong_Echo` passes RX flits through on port 1 and emits CETPH/CNP packets on port 2 when an incoming flit's NTH carries FECN. `UB_RTO_Timer` supports per-channel exponential backoff with a host-configurable base timeout.

Multi-flit Write payload. `UB_Eth_Encap` buffers header + payload bytes (up to 384 B for 256-byte writes) and emits the wire as 32-byte chunks; `UB_Eth_Decap` reassembles the wire stream and emits meta + ext + payload flits in sequence; `UB_HBM_Write` consumes the payload byte stream at incrementing HBM offsets. `UB_Jetty_Sched` and `UB_OrderTracker_Initiator` are multi-flit-packet-aware: non-sop continuation flits are queued under the same INI as the preceding sop, and a drain stays on one INI until the packet's eop is emitted. Verified end-to-end with `test_multi_flit_write.cpp` on five payload sizes (8/32/64/128/256 B).

Comparison-vs-baseline framing. OpenRoCE is intentionally an apples-to-apples baseline, not a fully optimised production RoCEv2 stack. It exists to anchor the comparison numbers on identical infrastructure (same OpenClickNP toolchain, same FPGA target, same evaluation harness). For the absolute-numbers question ("how does OpenURMA compare to NVIDIA ConnectX-7?"), we don't claim a meaningful answer — the comparison would require disentangling the protocol design from the microarchitectural optimisation budget that a vendor invests over many product cycles. The relative-numbers question ("how do UB's two architectural pillars trade against the standard RoCE design point in synthesisable RTL on identical infrastructure?") is the one we set out to answer.

Why no SRNIC / StaR / 1RMA / Aquila / Falcon / UEC / IRN / MELO / FaSR / DCP in-fabric comparison. A reviewer may reasonably ask why we did not run the connection-cache scheme of SRNIC [38], the stateless reconstruction of StaR [27], the connectionless-over-UDP design of 1RMA [35] or its Aquila-fabric deployment [7], the Falcon hardware transport [2], the Ultra Ethernet specification [37], or the IRN [30] / MELO [28] / FaSR [10] / DCP [24] retransmission schemes as additional baselines. The SRNIC, StaR, Aquila, and Falcon artifacts are vendor-internal silicon implementations; 1RMA's artifact is a software stack on top of a commodity RoCE NIC; UEC is a specification without a public RTL reference; IRN, MELO, FaSR, and DCP are published as algorithm-and-simulation or vendor-internal hardware studies with no public synthesisable FPGA RTL. None reduces to a synthesisable RTL stack on a commodity FPGA without effectively re-implementing it — which would be a separate paper of comparable scope per target. We treat those works as architectural reference points (§10) rather than as numerical baselines, and position OpenURMA against them along five orthogonal axes in Table 1. OpenRoCE is the single synthesisable in-fabric baseline because it is the only de-

sign point where we can confidently hold every variable below the protocol fixed.

10. Related Work

RDMA scalability has become one of the most active areas in datacenter networking, with substantial work across NSDI / SIGCOMM / OSDI / ATC in 2020–2025. We survey the field along seven axes and summarise OpenURMA's positioning in Table 1.

Connection-state scalability. The QP-scaling problem has driven a long line of work. FaSST [13] reduced the per-QP footprint via a UD verbs layer over fast packet processing. 1RMA [35] dropped per-connection state entirely by issuing one-sided RDMAs over UDP with explicit per-op authentication. Collie [17] characterised the RoCE-on-Ethernet performance cliff under PFC oscillation. SRNIC [38] proposed a connection cache atop a fixed-size on-chip QP context, spilling cold connections to host memory under hardware control. StaR [27] pushes the same axis further by reconstructing per-connection state on demand from host memory and packet metadata, so the NIC carries effectively zero per-QP context in steady state. Meta's production deployment at SIGCOMM'24 [6] gives the empirical face of the same problem: their distributed-training backend needs up to 32 QPs per source–destination pair to approach roofline throughput, exactly because RoCE RC's per-QP state pins each QP to a small set of paths. UB attacks the problem at the transport-layer *specification*, by making the connection a property of the host pair (TP Channel) rather than the application pair (QP); this is complementary to SRNIC's cache and StaR's reconstruction, since collapsing the cardinality of per-pair state also bounds whatever those techniques would otherwise have to spill or rebuild.

New reliable hardware transports. Aquila [7] co-designed a custom fabric with 1RMA cells to extend memory accesses across a clique without a host-pair connection. Google's Falcon [2], contributed to OCP in 2024 and presented at SIGCOMM'25, retains a per-connection abstraction but adds hardware SACK, fine-grained RTT-based shaping, and PSP-encrypted multi-path; it claims a $5\times$ op-rate and $10\times$ tail-latency improvement over the Pony Express software transport, and outperforms RoCE on its target workloads. Ultra Ethernet Specification 1.0 [37], released June 2025 by a $>$100-company Linux Foundation consortium, defines a packet-sprayed, SACK-based reliable transport with native RDMA semantics and scales to "millions of endpoints". UB is in the same architectural family — a clean-slate, connectionless, RDMA-class transport designed for AI/HPC workloads — but with a different ordering surface (graded §7.3 vs. UEC's per-transaction guarantees) and an explicitly open spec and (with OpenURMA) open RTL.

Programmable and FPGA-based transports. Tonic [1] introduced a Verilog-programmable transport-protocol framework on FPGA capable of expressing TCP, RoCE, and NDP variants in a unified pipeline. NanoTransport [12] provided a P4-programmable hardware transport prototype in Chisel/Firesim, with NDP and Homa demonstrations. StRoM [34] put a fully programmable RoCE-derived path on FPGA. Flor [23] is an open RDMA framework over heterogeneous RNICs. SwCC [8] adds an on-NIC RISC-V engine for software-programmable, per-packet congestion control. NICA [4] and AccelNet [5] build NIC offloads on programmable hardware. ClickNP [21] is the element-based FPGA substrate we build on through OpenClickNP [19], and KV-Direct [20] is a precedent for ClickNP-style decomposition applied to a specific RDMA workload. OpenURMA's contribution in this category is the first end-to-end UB data-path on commodity FPGA closing P&R at 322 MHz, not the FPGA-programmability story itself — our substrate is fixed-function once compiled.

Loss recovery and selective retransmission. IRN [30] is the canonical argument that an RDMA NIC should abandon Go-Back-N in favour of bitmap-tracked selective retransmission with per-packet ACKs, removing much of RoCE's reliance on PFC. MELO [28] sized the on-chip bookkeeping for scalable SR, using a shared bits pool to keep SACK metadata bounded regardless of connection count. FaSR [10] drives full-line-rate SR on the NIC with novel state-management schemes addressing the connection-vs-memory tension. DCP [24], the SIGCOMM'25 Best Student Paper Honorable Mention, presents an RTO-free, packet-LB-friendly hardware-offloadable SR design. LEFT [9] optimises packet reordering with a shared-bitmap pool and packet aggregation. OpenURMA's transport path implements both GBN and a selective `TPSACK`-driven retransmit slot (§3.4); we treat the above as algorithmic reference points, and a quantitative comparison under controlled loss injection (goodput-vs-loss-rate and retrans-buffer-occupancy curves) is left as a v4 deliverable consistent with the SW-emu loss-injection gap flagged in §6.

Multi-tenant performance isolation. A second scalability axis — orthogonal to per-pair state — is isolation between cotenants sharing the same NIC. Justitia [39] provides software multi-tenancy via userspace pacing and rate-limiting. Husky [16] characterises how RNIC microarchitecture resources (MTT/MPT cache, processing units) can be exhausted by adversarial workloads, showing SR-IOV and HW TC do not isolate at the microarchitectural level. Harmonic [26] adds hardware-assisted isolation for public-cloud RDMA. OpenURMA does not currently address this axis; UB's per-host-pair connection model nevertheless removes one of the cache-pressure sources Husky exploits, since the cache no longer scales with application-pair count.

Multi-path and adaptive routing. MP-RDMA [29] first observed that letting RDMA WRITE/READ packets traverse multiple paths and arrive out-of-order, with bitmap-tracked completion, recovers substantial multi-path bandwidth without breaking the workloads that consume it. ConWeave [36] extends this into the switch with in-network

Table 1: Design-point positioning vs. recent RDMA scalability work (2018–2025). Rows are grouped by primary contribution. "Conn. state" is what the NIC stores per peer pair / per connection; "Loss recov." is the on-NIC loss-recovery scheme; "Multi-path" is whether the transport admits multi-path delivery without reorder breakage; "Ordering surface" is what semantic ordering the spec exposes; "Substrate" is the realisation. UB/OpenURMA is the only point that combines per-host-pair connection state, a graded §7.3 ordering surface, and an open FPGA-RTL substrate.

Work	Conn. state	Loss recov.	Multi-path	Ordering surface	
RoCEv2 RC	per-QP	GBN	–	strict / QP	
FaSST [13]	UD (per msg)	app-level	–	none	SW +
1RMA [35]	none	per-op auth	yes	none	
Aquila + 1RMA [7]	cell, none	cell-level	cell-network	none	
SRNIC [38]	QP cache + spill	RoCE	–	strict / QP	
StaR [27]	reconstructed	RoCE	–	strict / QP	
Falcon [2]	per-conn	SACK + RTT	yes	per-flow	
UEC v1.0 [37]	packet-sprayed	SACK	yes	per-transaction	
Tonic [1]	programmable	programmable	programmable	programmable	F
NanoTransport [12]	programmable	programmable	programmable	programmable	FF
StRoM [34]	per-QP	RoCE-derived	–	per-QP	
Flor [23]	per-QP	RoCE	–	per-QP	o
SwCC [8]	per-QP	RoCE + SW CC	–	per-QP	
IRN [30]	per-QP	SR + per-pkt ACK	–	strict / QP	
MELO [28]	per-QP	const-mem SR	–	strict / QP	a
FaSR [10]	per-QP	line-rate SR	–	strict / QP	
DCP (SIGCOMM'25) [24]	per-QP	RTO-free SR	yes (pkt-LB)	strict / QP	
LEFT [9]	per-QP	shared bitmap	yes	strict / QP	
MP-RDMA [29]	per-QP + OOO	RoCE	yes	OOO + bitmap	
ConWeave [36]	per-QP	RoCE	yes	per-QP	
STrack [18]	per-flow	SR + fast recov	yes	per-flow	
Spectrum-X [33]	per-QP	RoCE	yes	per-QP	
Justitia [39]	per-QP	–	–	–	
Husky [16]	per-QP (diag)	–	–	–	
Harmonic [26]	per-QP	–	–	–	h
SCR [40]	SW-programmable	SW-prog	SW-prog	SW-prog	Bl
UB / OpenURMA	**per host-pair (TP Channel)**	**GBN + TPSACK**	**TPG + spray**	**graded §7.3 (UNO / NO / ROI / ROT / ROL / SO + Fence)**	**op**

reordering support for RoCE. STrack [18] is a hardware-offloaded multipath transport tuned for AI/ML clusters, combining adaptive load balancing, RTT-based CC, and SR-based loss recovery. Spectrum-X [33] adds adaptive-routing multi-path to commercial RoCE-on-Ethernet NICs. UB's TPG/spray [11] reuses the same architectural idea — spray flits across the path-diversity hash, recover order at the receiver — exposed as the default mode of the graded §7.3 ordering surface (UNO + NO); we implement TPG + spray in the TPG_Encoder/Decoder and Multipath_Steer elements (§3).

Software-controlled hardware transport. SCR / White-Boxing RDMA [40] is a complementary recent direction: it surfaces packet-granular software control over the BlueField-3 hardware transport via on-NIC Datapath Accelerators, enabling SW-defined CC, scheduling, and multi-path on top of vendor RoCE silicon. OpenURMA targets a different trade — the entire transport is RTL on FPGA, with no SW fast-path involvement — but the two are not mutually exclusive: a future v4 could expose a similar SW-programmable surface on top of OpenURMA's open RTL.

Datacenter transports and congestion control. DCQCN [41] is the de-facto RoCE congestion control; we mirror it as the OpenRoCE baseline's CC. UB's C-AQM [11] is a queue-occupancy-based ECN signal; we model it as a SystemC + SW-emu element so the closed-loop behaviour is testable end-to-end without a physical switch.

Ordering semantics: total order vs. relaxed order. At the opposite end of the ordering design space from URMA, 1Pipe [22] provides a causally and totally ordered communication abstraction across a whole datacenter, with the goal of simplifying distributed transactions, state-machine replication, and distributed atomic operations. 1Pipe pays for the guarantee with in-network barrier aggregation at every switch and an extra $\sim$0.5–1 RTT of barrier-wait latency on each message, with reliable scatterings adding a further RTT for 2PC commit. URMA traffic is the wrong workload for that trade. The bulk of URMA verbs — ML training collectives, KV-cache / tensor transfers, parameter-server fan-in/fan-out, distributed checkpoint — are bandwidth-bound memory operations whose correctness contract is scoped to a single (Initiator, target Jetty) pair; cross-host total order is not part of the contract and paying for it costs real RTTs on the critical path. URMA's §7.3 graded surface (§2.3) is the specification-level descendant of MP-RDMA's relaxed-ordering insight discussed above: UNO + NO is the default fast path (no per-INI ordering at all, the regime where MP-RDMA's multi-path delivery is safe by construction), ROI / ROT / ROL graduate strictly more ordering as the workload demands it, and SO + Fence is the escape

hatch for the rare verb that does need strict per-WR serialization. 1Pipe-style total order remains a good fit for the higher-level distributed-transaction layer that may sit on top of URMA — not for the verb path itself, which is what this paper implements.

Adjacent fabrics. NVLink [31] and NVLink Fusion are NVIDIA-internal fabric protocols whose specification is partially open. CXL [3] is a load/store fabric over PCIe physical layers, not an RDMA-class network. UB and OpenURMA target the same scale-out / DC-network niche as RoCE/IB, not CXL's intra-rack memory-coherence niche.

11. Conclusion

OpenURMA puts UB's two architectural moves — collapsing connection state from per-application-pair to per-host-pair, and exposing ordering as a graded surface rather than a single guarantee — into synthesisable RTL on a commodity FPGA. The implementation closes 322 MHz Vivado P&R across all 38 elements; the parallel 21-element OpenRoCE baseline closes the same target on the same toolchain.

Three observations shape the takeaway. First, the per-host-pair connection model converts the $N{\cdot}M$ state explosion into $N{+}M$ growth, and the gap is asymptotic, not constant: it widens from 5× at $(N{=}M{=}1)$ to 4,855× at $(N{=}M{=}1024)$, which is the regime where the design moves NIC context from on-chip BRAM to spill-to-host-DRAM for RoCE while OpenURMA still fits on chip. Second, the graded §7.3 ordering surface is cheap to implement: 13 KLUT (10.6% of the design) and *zero* extra pipeline cycles for paths that do not request ordering — the cost is paid only by workloads that opt in. Third, the price of the two pillars combined is bounded: $\sim$14% of a U50's LUT budget and 46.6 ns of pipeline depth relative to the matched RC reference; both small against the network RTT ($\sim \mu$s) on the inter-host path the transport is designed for.

The artefact is intended as a substrate for further work: studying loss-recovery policies under multi-flit retransmission, alternative congestion signals, and multi-path scheduling, with synthesisable numbers rather than simulation alone; and as a controlled baseline against which other connectionless or graded-ordering transports (Falcon, UEC, SRNIC/StaR derivatives) can be compared on identical infrastructure.

Code. `https://github.com/bojieli/OpenURMA`.

Acknowledgments. This work builds on the OpenClickNP toolchain (an open re-implementation of the ClickNP element model) and the published UB-Base-Specification 2.0.1.

References

[1] Mina Tahmasbi Arashloo, Alexey Lavrov, Manya Ghobadi, Jennifer Rexford, David Walker, and David Wentzlaff. Enabling programmable transport protocols in high-speed NICs. In *Proc. USENIX NSDI*, 2020.

[2] Google Authors. Falcon: A reliable, low latency hardware transport. In *Proc. ACM SIGCOMM*, 2025. Specification contributed to OCP 2024; please replace placeholder author list with the published list before camera-ready.

[3] CXL Consortium. Compute Express Link (CXL) Specification 3.1. `https://www.computeexpresslink.org/`, 2024.

[4] Haggai Eran, Lior Zeno, Maroun Tork, Gabi Malka, and Mark Silberstein. NICA: An infrastructure for inline acceleration of network applications. In *Proc. USENIX ATC*, 2019.

[5] Daniel Firestone, Andrew Putnam, Sambhrama Mundkur, Derek Chiou, Alireza Dabagh, Mike Andrewartha, Hari Angepat, et al. Azure Accelerated Networking: SmartNICs in the public cloud. In *Proc. USENIX NSDI*, 2018.

[6] Adithya Gangidi, Rui Miao, Shengbao Zheng, Sai Jayesh Bondu, Guilherme Goes, Hany Morsy, Rohit Puri, Mohammad Riftadi, Ashmitha Jeevaraj Shetty, Jingyi Yang, Shuqiang Zhang, Mikel Jimenez Fernandez, Shashidhar Gandham, and Hongyi Zeng. RDMA over Ethernet for distributed training at meta scale. In *Proc. ACM SIGCOMM*, 2024.

[7] Dan Gibson, Hema Hariharan, Eric Lance, Moray McLaren, Behnam Montazeri, Arjun Singh, Stephen Wang, Hassan M. G. Wassel, Zhehua Wu, Sunghwan Yoo, Raghuraman Balasubramanian, Prashant Chandra, Michael Cutforth, Peter Cuy, David Decotigny, Rakesh Gautam, Alex Iriza, Milo M. K. Martin, Rick Roy, Zuowei Shen, Ming Tan, Ye Tang, Monica Wong-Chan, Joe Zbiciak, and Amin Vahdat. Aquila: A unified, low-latency fabric for datacenter networks. In *Proc. USENIX NSDI*, 2022.

[8] Hongjing Huang et al. SwCC: Software-programmable and per-packet congestion control for RDMA. In *Proc. USENIX ATC*, 2025. Authors per public record; please verify before camera-ready.

[9] Peihao Huang et al. LEFT: Lightweight and fast packet reordering for RDMA. In *Proc. APNet*, 2024. Authors per public record; please verify before camera-ready.

[10] Xinyang Huang, Kai Chen, et al. Fast and scalable selective retransmission for RDMA. In *Proc. IEEE conf.*, 2024. Authors / venue per IEEE Xplore record; please verify before camera-ready.

[11] Huawei Technologies. UB-base-specification 2.0.1. `https://www.unifiedbus.org/`, 2024. Unified Bus consortium specification, available from the consortium's documentation portal.

[12] Stephen Ibanez, Alex Mallery, Serhat Arslan, Theo Jepsen, Muhammad Shahbaz, Nick McKeown, and Changhoon Kim. NanoTransport: A low-latency, programmable transport layer for NICs. In *Proc. ACM SOSR*, 2021.

[13] Anuj Kalia, Michael Kaminsky, and David G. Andersen. Using RDMA efficiently for key-value services. In *Proc. ACM SIGCOMM*, 2014.

[14] Anuj Kalia, Michael Kaminsky, and David G. Andersen. FaSST: Fast, scalable and simple distributed transactions with two-sided (RDMA) datagram RPCs. In *Proc. USENIX OSDI*, 2016.

[15] Anuj Kalia, Michael Kaminsky, and David G. Andersen. Datacenter RPCs can be general and fast. In *Proc. USENIX NSDI*, 2019.

[16] Xinhao Kong, Jingrong Chen, Wei Bai, Yechen Xu, Mahmoud Elhaddad, Shachar Raindel, Jitendra Padhye, Alvin R. Lebeck, and Danyang Zhuo. Understanding RDMA microarchitecture resources for performance isolation. In *Proc. USENIX NSDI*, 2023.

[17] Xinhao Kong, Yibo Zhu, Huaping Zhou, Zhuo Jiang, Jianxi Ye, Chuanxiong Guo, and Danyang Zhuo. Collie: Finding performance anomalies in RDMA subsystems. In *Proc. USENIX NSDI*, 2022.

[18] Yanfang Le et al. STrack: A reliable multipath transport for AI/ML clusters. In *arXiv preprint*, 2024. Authors per public record; please verify before camera-ready.

[19] Bojie Li. OpenClickNP: a clean-room reimplementation of ClickNP on Alveo U50. `https://github.com/bojieli/OpenClickNP`, 2025–2026.

[20] Bojie Li, Zhenyuan Ruan, Wencong Xiao, Yuanwei Lu, Yongqiang Xiong, Andrew Putnam, Enhong Chen, and Lintao Zhang. KV-Direct: High-performance in-memory key-value store with programmable NIC. In *Proc. ACM SOSP*, 2017.

[21] Bojie Li, Kun Tan, Layong Larry Luo, Yanqing Peng, Renqian Luo, Ningyi Xu, Yongqiang Xiong, Peng Cheng, and Enhong Chen. ClickNP: Highly flexible and high-performance network processing with reconfigurable hardware. In *Proc. ACM SIGCOMM*, 2016.

[22] Bojie Li, Gefei Zuo, Wei Bai, and Lintao Zhang. 1Pipe: Scalable total order communication in data center networks. In *Proc. ACM SIGCOMM*, 2021.

[23] Qiang Li et al. Flor: An open high performance RDMA framework over heterogeneous RNICs. In *Proc. USENIX OSDI*, 2023. Authors per public record; please verify before camera-ready.

[24] Wenxue Li, Xiangzhou Liu, Yunxuan Zhang, Zihao Wang, Wei Gu, Tao Qian, Gaoxiong Zeng, Shoushou Ren, Xinyang Huang, Zhenghang Ren, Bowen Liu, Junxue Zhang, Kai Chen, and Bingyang Liu. Revisiting RDMA reliability for lossy fabrics. In *Proc. ACM SIGCOMM*, 2025. Best Student Paper, Honorable Mention.

[25] Xiaofeng Lin, Yu Chen, Xiaodong Li, et al. Fastsocket: An almost drop-in replacement for the Linux socket interface for High-Performance Networking. In *Proc. USENIX ATC*, 2017.

[26] Zhuolong Lou et al. Harmonic: Hardware-assisted RDMA performance isolation for public clouds. In *Proc. USENIX NSDI*, 2024. Authors per public record; please verify before camera-ready.

[27] Jie Lu, Jiajun Hu, Tianyin Xu, Jianbo Dong, Liang Liu, Hongqiang Harry Liu, Zilong Wang, and Kai Chen. Towards stateless RNIC for data center networks. In *Proc. USENIX NSDI*, 2024. Authors / venue per public record; please verify before camera-ready.

[28] Yuanwei Lu, Guo Chen, Bojie Li, Kun Tan, Yongqiang Xiong, Peng Cheng, Jiansong Zhang, Enhong Chen, and Thomas Moscibroda. Memory efficient loss recovery for hardware-based transport in datacenter. In *Proc. APNet*, 2017.

[29] Yuanwei Lu, Guo Chen, Bojie Li, Kun Tan, Yongqiang Xiong, Peng Cheng, Jiansong Zhang, Enhong Chen, and Thomas Moscibroda. Multi-Path transport for RDMA in datacenters. In *Proc. USENIX NSDI*, 2018.

[30] Radhika Mittal, Alexander Shpiner, Aurojit Panda, Eitan Zahavi, Arvind Krishnamurthy, Sylvia Ratnasamy, and Scott Shenker. Revisiting network support for RDMA. In *Proc. ACM SIGCOMM*, 2018.

[31] NVIDIA Corporation. NVLink: A high-bandwidth inter-GPU interconnect. Vendor whitepaper, 2014–2025. Successive generations of the NVLink fabric are described in the NVIDIA whitepaper series.

[32] NVIDIA Corporation. NVIDIA BlueField-3 DPU datasheet. NVIDIA Networking product brief, 2023. Available from NVIDIA's data-processing-unit product page.

[33] NVIDIA Networking. NVIDIA Spectrum-X: Adaptive routing and telemetry-based congestion control for AI networks. NVIDIA technical brief, 2024. Vendor description of multi-path adaptive-routing delivery over Spectrum-4 / BlueField-3 NICs; the closest commercially-deployed point of comparison to UB's TPG multi-path scheme.

[34] David Sidler, Zeke Wang, Monica Chiosa, Amit Kulkarni, and Gustavo Alonso. StRoM: Smart remote memory. In *Proc. EuroSys*, 2020.

[35] Arjun Singhvi, Aditya Akella, Dan Gibson, Thomas F. Wenisch, Monica Wong-Chan, Sean Clark, Milo M. K. Martin, Moray McLaren, Prashant Chandra, Rob Cauble, et al. 1RMA: Re-envisioning remote memory

access for multi-tenant datacenters. In *Proc. ACM SIGCOMM*, 2020.

[36] Cha Hwan Song, Xin Zhe Khooi, Raj Joshi, Inho Choi, Jialin Li, and Mun Choon Chan. Network load balancing with in-network reordering support for RDMA. In *Proc. ACM SIGCOMM*, 2023.

[37] Ultra Ethernet Consortium. Ultra Ethernet specification 1.0. Industry specification, 2025. Released June 2025 under Linux Foundation JDF; `https://ultraethernet.org/`.

[38] Zilong Wang, Layong Luo, Qingsong Ning, Chaoliang Zeng, Wenxue Li, Xinchen Wan, Peng Xie, Tao Feng, Ke Cheng, Xiongfei Geng, Tianhao Wang, Weicheng Ling, Kejia Huo, Pingbo An, Kui Ji, Shideng Zhang, Bin Xu, Ruiqing Feng, Tao Ding, Kai Chen, and Chuanxiong Guo. SRNIC: A scalable architecture for RDMA NICs. In *Proc. USENIX NSDI*, 2023.

[39] Yiwen Zhang, Yue Tan, Brent Stephens, and Mosharaf Chowdhury. Justitia: Software multi-tenancy in hardware kernel-bypass networks. In *Proc. USENIX NSDI*, 2022.

[40] Chenxingyu Zhao, Jaehong Min, Ming Liu, and Arvind Krishnamurthy. White-boxing RDMA with packet-granular software control. In *Proc. USENIX NSDI*, 2025.

[41] Yibo Zhu, Haggai Eran, Daniel Firestone, Chuanxiong Guo, Marina Lipshteyn, Yehonatan Liron, Jitendra Padhye, Shachar Raindel, Mohamad Haj Yahia, and Ming Zhang. Congestion control for Large-Scale RDMA deployments. In *Proc. ACM SIGCOMM*, 2015.